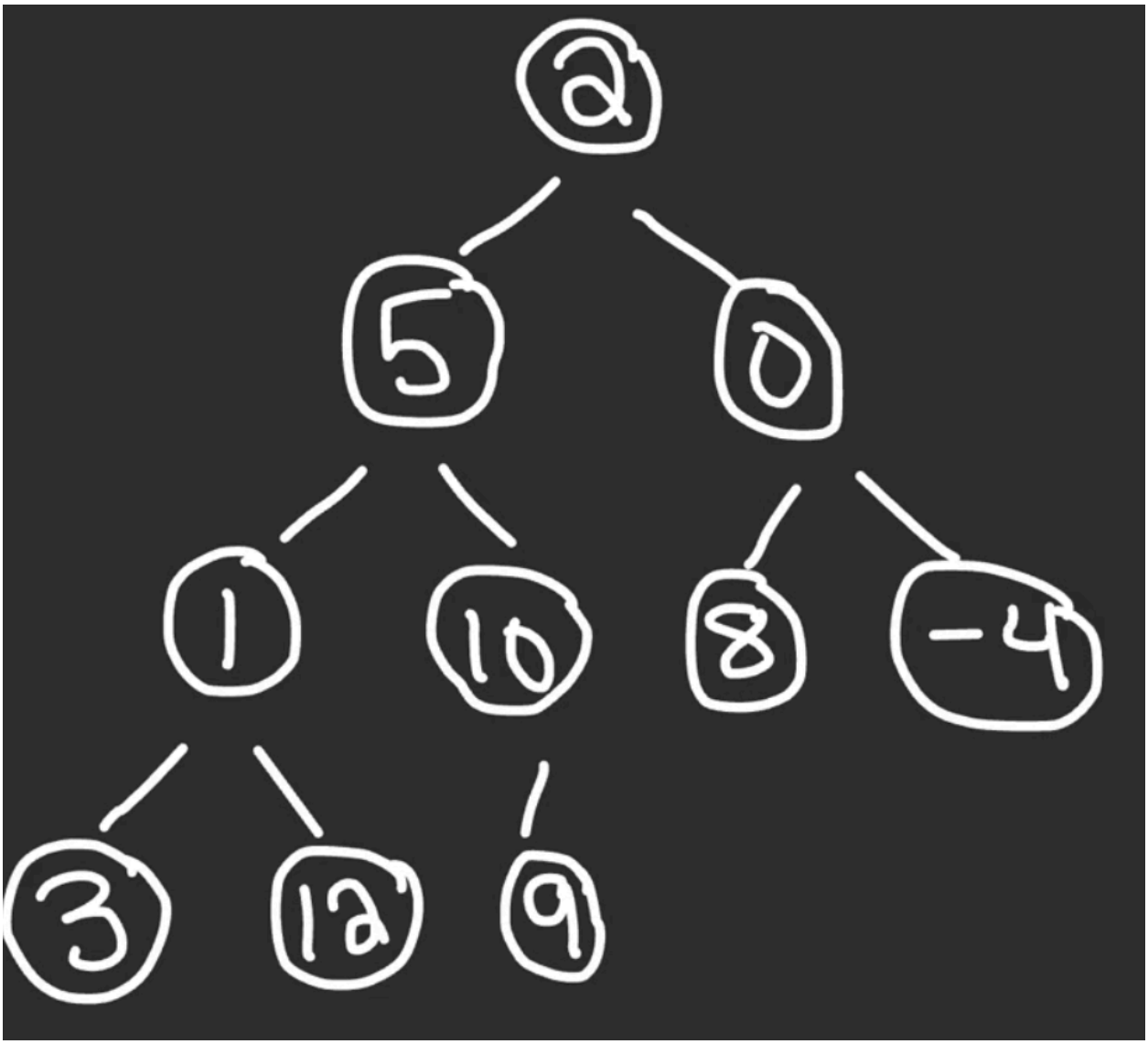
Heap

→ Heap is a particular type of binary tree : Minheap and Maxheap (two kinds)



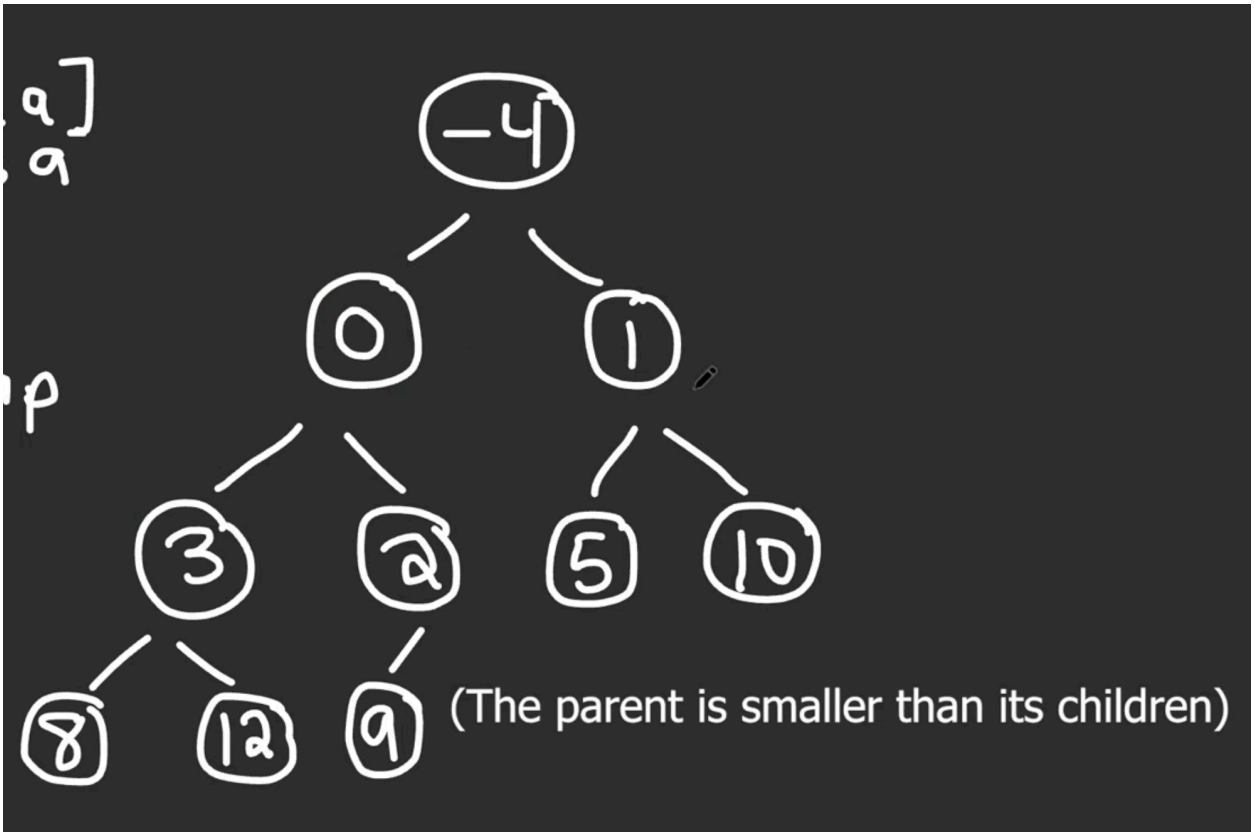
→ Heaps can be represnted in arrays using :

- Start with the root at i=0
- The left child of i node is 2i+1
- The right child of i node is 2i+2

So we have A=[2, 5, 0, 1, 10, 8, -4, 3, 12, 9]

MinHeap

→ Every element at i is smaller than the its children



Operations :

1. Heap pop (O(logn)) : Remove the mimum element and rearrage such that it remains a MinHeap
2. Heap push O(logn)) : Insert new element and rearrange such that it remains a MinHeap.
3. Heap peek: O(1) : Read the minimum element (first element of the array)
4. Hear sort (O(nlogn)): Sort the elements in min to max order (repeated pop and arrange)
5. Heapify i.e building the heap (O(n)) : Given an array make it into a minHeap

Same operations for MaxHeap

Using the heapq library in python

```
import heapq

A=[-4, 3, 1, 0, 2, 5, 10, 8, 12, 9]

# Building a MinHeap (heapfiy)
heapq.heapify(A)  # Makes it : A=[-4, 0, 1, 3, 2, 5, 10, 8, 1
2, 9]

# Pushing an element
heapq.heappush(A, 4) # Makes it : A=[-4, 0, 1, 3, 2, 5, 10,
8, 12, 9, 4]

# Popping an element
minn=heapq.heappop(A) # Makes it : A=[0, 2, 1, 3, 4, 5, 10,
8, 12, 9]
                                                # minn=-4

# Sorting the heap
def heapsort(arr):
    heap.heapify(arr)
    n=len(arr)
    new_list=[0]*n

    for i in range(n):
        minn=heap.heappop(arr)
        new_list[i]=minn

    return new_list
```

```
'''
heapq by deafult supports only minheap, to make it max heap
(workaround):
'''

# Building a MaxHeap(heapify)
B=[-4, 3, 1, 0, 2, 5, 10, 8, 12, 9]
n=len(B)

for i in range(B):
    B[i]= -B[i] # Multiplying by negative to make minheap beh
ave like maxheap

heapq.heapify(B) # Makes it : B=[-12, -9, -10, -8, -2, -5, -
1, -3, 0, 4]

'''
Although signs are reversed, so when popping or any other ope
ration, handle it.
'''
```